BCS702 - Parallel Computing

Module 5 - GPU Programming with CUDA

Dec 2025 / Jan 2026 Question Paper - Q.9 and Q.10 with Model Answers

Q. No.	Question	Marks
Q.9 (a)	Define GPU and GPGPU. Explain the need for GPGPU.	6
Q.9 (b)	Explain thread, block and grid in CUDA.	6
Q.9 (c)	Explain CUDA vector addition program with suitable example.	8
Q.10 (a)	Compare CUDA and OpenCL.	6
Q.10 (b)	Explain Kernel with shared memory.	6
Q.10 (c)	Explain Heterogeneous computing in detail.	8

Writing tip: write the question number, a short **introduction**, then the headings given here. Draw the **diagrams**, write the **syntax / code** neatly, and end with a **conclusion**. Underline the **bold keywords**. Answer **Q.9 or Q.10** (all three parts) from this module.

Q. 9 (a) Define GPU and GPGPU. Explain the need for GPGPU. [6 Marks]

Introduction

In the late 1990s and early 2000s the computer industry responded to the demand for highly realistic **video games and video animations** by developing extremely powerful **graphics processing units (GPUs)**.

Definition of GPU

A **GPU (Graphics Processing Unit)** is a processor designed to **improve the performance of programs that need to render many detailed images**.

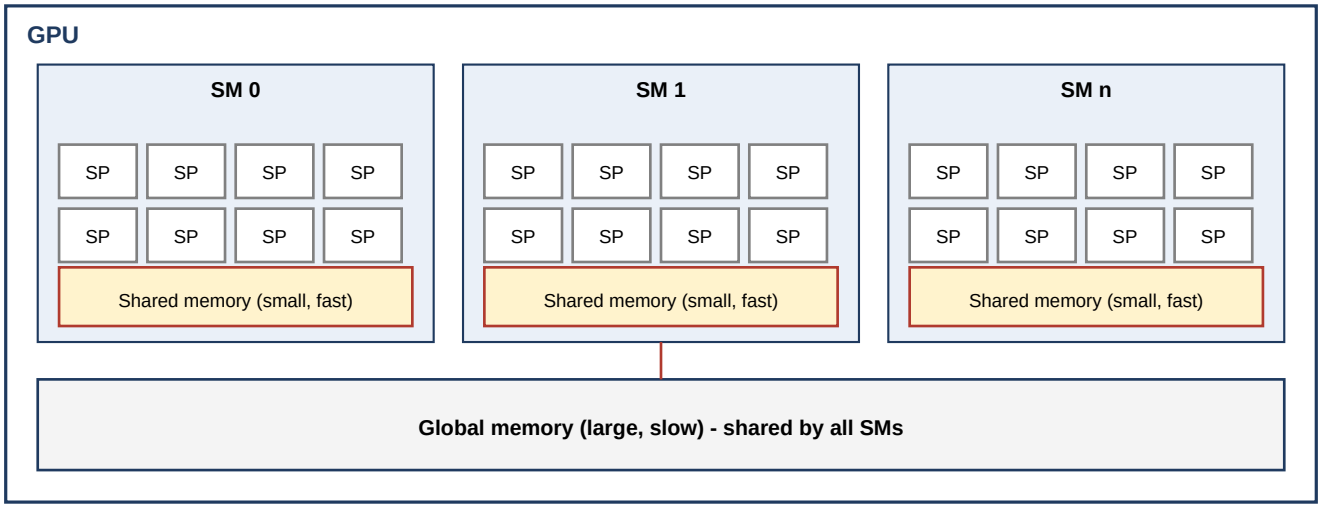
Definition of GPGPU

By the early 2000s, programmers who did not specialise in graphics tried to apply the computational power of GPUs to **general computational problems** such as **searching and sorting**, rather than graphics. This became known as **General Purpose computing on GPUs (GPGPU)**.

Early difficulties of GPGPU

- GPUs could be programmed only with **graphics APIs** such as **Direct3D and OpenGL**.
- Programmers had to **reformulate** general algorithms using graphics concepts such as **vertices, triangles and pixels**, which added considerable complexity.
- Groups therefore developed languages and compilers that allow general algorithms to be written in APIs resembling conventional high-level languages. The most widely used APIs now are **CUDA** (for Nvidia GPUs) and **OpenCL** (portable).

GPU architecture (basis of the need for GPGPU)



- A GPU is built from **SIMD (Single Instruction stream, Multiple Data stream)** processors: one **control unit** fetches an instruction and broadcasts it to many **datapaths**; each datapath either executes the instruction on its data or is **idle**.
- Nvidia GPUs are composed of **Streaming Multiprocessors (SMs)**. Each SM has several control units and many datapaths called **Streaming Processors (SPs)** (cores).
- The SMs operate **asynchronously**: no penalty if one branch of an `if-else` runs on one SM and the other branch on another SM.
- Nvidia calls this **SIMT** (Single Instruction Multiple Thread): threads executing the same instruction may not execute simultaneously, because some threads may block on memory access while others proceed (hides memory latency).

- Each SM has a small, fast **shared memory** shared among its SPs; all SMs share a much larger but slower **global memory**.
- The GPU and its memory are physically separate from the CPU and its memory: the CPU with its memory is the **host**, the GPU with its memory is the **device**.

Example of branching on a SIMD system (`if (x[i] >= 0) x[i] += 1; else x[i] -= 2;`):

Time	Datapaths with x[i] >= 0	Datapaths with x[i] < 0
1	Test `x[i] >= 0`	Test `x[i] >= 0`
2	`x[i] += 1`	Idle
3	Idle	`x[i] -= 2`

If the two groups run on **different SMs**, the `if-else` needs only two stages instead of three.

Need for GPGPU

1. **Slowdown of CPU performance:** from about 1986 to 2003 single-thread performance of CPUs grew by more than **50% per year**, but from 2015 to 2017 it grew by less than **4% per year**. Programmers therefore look for other types of processors to boost performance, and GPUs are one possibility.
2. **Massive parallelism:** one of the most powerful Nvidia processors has **82 SMs with 128 SPs each = 10,496 SPs**.
3. **Suited to data-parallel problems** such as adding two vectors, searching and sorting, where the same operation is applied to many data items.
4. **Easier programming:** APIs like CUDA let programmers write general-purpose programs without graphics concepts.
5. It leads to **heterogeneous computing**, in which the CPU (host) and the GPU (device) cooperate in one program.

Conclusion

A GPU is a powerful graphics processor; GPGPU uses it for general computation. Slow growth of CPU performance and the large number of SPs on a GPU created the need for GPGPU.

Q. 9 (b) Explain thread, block and grid in CUDA. [6 Marks]

Introduction

An Nvidia GPU consists of **streaming multiprocessors (SMs)**, and each SM consists of **streaming processors (SPs)**. When a CUDA kernel runs, each individual **thread** executes its code on an SP. CUDA organises threads into **blocks** and **grids**.

Thread

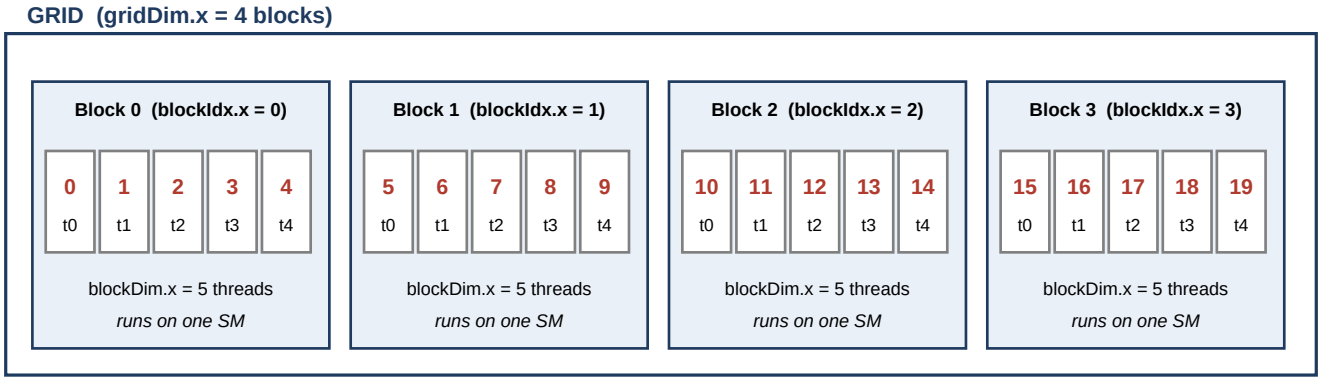
- A **thread** is one copy of the kernel code executing on an SP.
- All threads run the same code on their own data (**SPMD**). The thread-specific data is its rank, stored in `threadIdx.x`.

Thread block (block)

- A **thread block** is a collection of threads that run on a **single SM**.
- In the kernel call `Kernel<<<blk_ct, th_per_blk>>>(...)` the **first value is the number of thread blocks** and the **second is the number of threads in each block**.
- `Hello<<<1, thread_count>>>()` uses **one block**, so only one SM. On a GPU with two SMs, `Hello<<<2, thread_count/2>>>()` divides the threads between the two SMs.

Grid

A **grid** is the collection of thread blocks started by a kernel. So a thread block is composed of threads, and a grid is composed of thread blocks.



Built-in variables

The following four variables are structs initialised in each thread's memory when a kernel starts. Each has three **unsigned integer** fields **x, y and z**.

Variable	Meaning
`threadIdx`	Rank or index of the thread in its thread block
`blockDim`	Dimensions (shape / size) of the thread blocks
`blockIdx`	Rank or index of the block within the grid
`gridDim`	Dimensions (shape / size) of the grid

Initialisation of gridDim and blockDim

With `Hello<<<blk_ct, th_per_blk>>>()` the values in angle brackets are assigned to the x fields: `gridDim.x = blk_ct; blockDim.x = th_per_blk;` The y and z fields are set to **1**. To use other values, declare variables of type **dim3**:

```
dim3 grid_dims, block_dims;
grid_dims.x = 2;  grid_dims.y = 3;  grid_dims.z = 1;
block_dims.x = 4; block_dims.y = 4; block_dims.z = 4;
Kernel<<<grid_dims, block_dims>>>(...);
```

This starts a grid with 2 x 3 x 1 = **6 blocks**, each with 4 x 4 x 4 = **64 threads**.

Global rank of a thread

```
rank = blockDim.x * blockIdx.x + threadIdx.x
```

For 4 blocks with 5 threads per block, the global ranks are:

blockIdx.x	t=0	t=1	t=2	t=3	t=4
0	0	1	2	3	4
1	5	6	7	8	9
2	10	11	12	13	14
3	15	16	17	18	19

Example program

```
__global__ void Hello(void) {
    printf("Hello from thread %d in block %d\n", threadIdx.x, blockIdx.x);
}
int main(int argc, char* argv[]) {
    int blk_ct = strtol(argv[1], NULL, 10);       /* number of blocks   */
    int th_per_blk = strtol(argv[2], NULL, 10);   /* threads per block  */
    Hello<<<blk_ct, th_per_blk>>>();              /* blk_ct*th_per_blk threads */
    cudaDeviceSynchronize();                      /* wait for GPU */
    return 0;
}
```

Important rules and limits

- All blocks must have the **same dimensions**.
- Thread blocks must be **independent**: a block must be able to complete regardless of the state of other blocks; blocks may execute **sequentially in any order or in parallel**. The GPU can then schedule a block using only its own state.
- Limits depend on the **compute capability** (a.b). For capability > 1, max **1024 threads per block**; max threads per SM is 1536 (2.b) and 2048 (> 2); max x- or y-dimension 1024 and z-dimension 64.

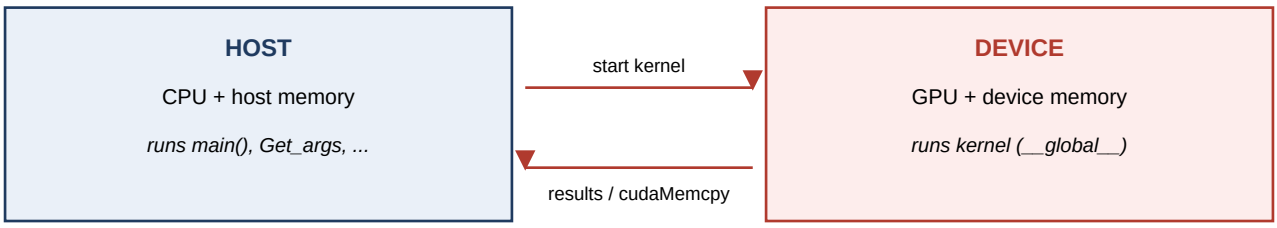
Conclusion

Threads are grouped into blocks (each run on one SM), and blocks form a grid. The built-in variables `threadIdx`, `blockIdx`, `blockDim` and `gridDim` let each thread find its own work.

Q. 9 (c) Explain CUDA vector addition program with suitable example. [8 Marks]

Introduction

GPUs and CUDA are especially effective for **data-parallel** programs. Vector addition is "embarrassingly parallel": define three n-element arrays `x`, `y`, `z` (`float`, since GPUs have more 32-bit than 64-bit floating-point units). `x` and `y` are initialised on the host; a kernel starts **at least n threads** and the i-th thread computes `z[i] = x[i] + y[i];`



Program (kernel and main)

```
__global__ void Vec_add(const float x[], const float y[], float z[], const int n) {
    int my_elt = blockDim.x * blockIdx.x + threadIdx.x;   /* global rank */
    /* total threads = blk_ct*th_per_blk may be > n */
    if (my_elt < n)
        z[my_elt] = x[my_elt] + y[my_elt];
}

int main(int argc, char* argv[]) {
    int n, th_per_blk, blk_ct;
    char i_g;                       /* are x and y user input or random? */
    float *x, *y, *z, *cz;
    double diff_norm;

    Get_args(argc, argv, &n, &blk_ct, &th_per_blk, &i_g);
    Allocate_vectors(&x, &y, &z, &cz, n);
    Init_vectors(x, y, n, i_g);

    Vec_add<<<blk_ct, th_per_blk>>>(x, y, z, n);   /* invoke kernel */
    cudaDeviceSynchronize();                        /* wait for it   */

    Serial_vec_add(x, y, cz, n);                    /* check result  */
    diff_norm = Two_norm_diff(z, cz, n);
    printf("Two-norm of difference between host and device = %e\n", diff_norm);

    Free_vectors(x, y, z, cz);
    return 0;
}
```

Explanation

1. The kernel

- Each thread finds its **global rank** `my_elt = blockDim.x * blockIdx.x + threadIdx.x` (total threads = `gridDim.x * blockDim.x`) and uses it as the array subscript.
- The total number of threads may not equal `n`, so the check `my_elt < n` is made. Example: for n = 997 (prime) we may start 4 blocks of 256 threads = 1024; the last 27 threads skip the addition.
- The kernel is the serial `for` loop with each iteration assigned to a different thread. With Foster's method, tasks are the individual additions, so no communication or aggregation is needed and mapping assigns each addition to a thread.

2. Get_args (host, standard C)

Reads `n`, `blk_ct`, `th_per_blk` and `i_g` from the command line. Prints a usage message and exits if the arguments are wrong or if `n` is greater than `blk_ct * th_per_blk`.

3. Allocate_vectors and managed memory

```
cudaMallocManaged(x_p, n*sizeof(float));   /* used on host and device */
cudaMallocManaged(y_p, n*sizeof(float));
cudaMallocManaged(z_p, n*sizeof(float));
*cz_p = (float*) malloc(n*sizeof(float));   /* cz used only on host    */
```

- `cudaMallocManaged` allocates **unified memory**: pointers can be used on both host and device as if they shared one memory, even though physical memories are separate.
- `cz` is used only on the host (to check the answer), so ordinary `malloc` is used.
- Cautions: needs compute capability >= 3.0 and a 64-bit host OS; on devices with capability < 6.0 the host and device cannot access it simultaneously (kernel has exclusive access); kernels using unified memory can be slower, because the system decides when to transfer data.

4. Other functions

- `Init_vectors`: reads `x`, `y` with `scanf` or generates them with `random`, depending on `i_g`.
- `Serial_vec_add`: adds `x` and `y` on the host with a `for` loop into `cz`.
- `Two_norm_diff`: sqrt of the sum of squares of `(z[i] - cz[i])`; it should be (almost) zero if the kernel is correct.
- `Free_vectors`: `cudaFree` for managed arrays, `free` for `cz`.

5. Execution flow

1. Host starts in `main`, reads arguments, allocates and initialises vectors.
2. `Vec_add<<<blk_ct, th_per_blk>>>` starts the kernel. Kernel execution is **asynchronous**.
3. `cudaDeviceSynchronize()` makes the host wait until all threads finish.
4. Host verifies the result, frees memory and terminates.

Version without unified memory (explicit transfers)

The kernel is unchanged. Separate host arrays `hx, hy, hz` (`malloc`) and device arrays `dx, dy, dz` (`cudaMalloc`) are used, and data is copied with `cudaMemcpy`:

```
cudaMemcpy(dx, hx, n*sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(dy, hy, n*sizeof(float), cudaMemcpyHostToDevice);
Vec_add<<<blk_ct, th_per_blk>>>(dx, dy, dz, n);
cudaMemcpy(hz, dz, n*sizeof(float), cudaMemcpyDeviceToHost);
```

`cudaMemcpy` is **synchronous**, so it waits for the kernel to finish, and `cudaDeviceSynchronize` is not needed. This version works on any device but the programmer must manage transfers.

Conclusion

In CUDA vector addition each thread adds one pair of elements, identified by its global rank. The program shows kernel launch, unified or explicit memory handling, synchronisation and verification.

Q. 10 (a) Compare CUDA and OpenCL. [6 Marks]

Introduction

Early GPGPU programmers had to use graphics APIs (Direct3D, OpenGL). Several groups then developed APIs that resemble conventional high-level languages. **Currently, the most widely used APIs for general-purpose programming on GPUs are CUDA and OpenCL.**

CUDA

- A software platform for writing GPGPU programs for **heterogeneous systems equipped with an Nvidia GPU**; developed for use **only on Nvidia GPUs**.
- Originally "Compute Unified Device Architecture"; Nvidia now says CUDA is just the name of an API.
- Language-specific APIs exist for **C, C++, Fortran, Python and Java**. The textbook uses **CUDA C**.
- Because it runs only on Nvidia GPUs it needs **relatively modest setup** code.
- CUDA is **not just a library**; it needs a **special compiler (`nvcc`)** that produces machine code for two processors, the host CPU and the device GPU. Files have the `.cu` suffix.
- Kernels are marked `__global__` and started with `<<<blocks, threads>>>`. Provides unified memory (`cudaMallocManaged`) and explicit transfers (`cudaMalloc`, `cudaMemcpy`).

A small CUDA program (to show the CUDA style)

```
#include <stdio.h>
#include <cuda.h>                       /* header file for CUDA */

__global__ void Hello(void) {           /* kernel: runs on GPU */
    printf("Hello from thread %d!\n", threadIdx.x);
}
int main(int argc, char* argv[]) {      /* host code: runs on CPU */
    int thread_count = strtol(argv[1], NULL, 10);
    Hello<<<1, thread_count>>>();       /* start thread_count threads on GPU */
    cudaDeviceSynchronize();            /* wait for GPU to finish */
    return 0;
}
```

- The file has a **.cu** suffix and is compiled with `$ nvcc -o cuda_hello cuda_hello.cu`; run with `$ ./cuda_hello 10`.
- A kernel is a function **started by the host but run on the device**; it is marked `__global__` and has return type `void`.
- `<<<1, thread_count>>>` gives the number of **thread blocks** and the **threads per block**.
- Kernel execution is **asynchronous**, so `cudaDeviceSynchronize()` makes the host wait for the GPU.
- Requires an Nvidia GPU of suitable **compute capability** (for example >= 2.0 for this program; unified memory needs >= 3.0).

CUDA execution model (used by Nvidia GPUs)

- A GPU is a collection of **streaming multiprocessors (SMs)**, each with many **streaming processors (SPs)** that act as SIMD datapaths (Nvidia calls this **SIMT**).
- Threads are grouped into **blocks** (one SM each) and **grids**; thread blocks must be independent of each other.
- Each SM has fast **shared memory**; all SMs share slower **global memory**.
- Limits depend on **compute capability** (a.b): e.g. maximum 1024 threads per block for capability > 1. Architectures include Tesla (1.b), Fermi (2.b), Kepler (3.b), Maxwell (5.b), Pascal (6.b), Volta (7.0), Turing (7.5) and Ampere (8.0).

OpenCL

- Designed to be **highly portable**: for use on **arbitrary GPUs and other processors**, such as **FPGAs (field programmable gate arrays)** and **DSPs (digital signal processors)**.
- To ensure this portability, an OpenCL program must include **a good deal of code** giving information about **which systems it can run on** and **how it should be run**.

Comparison

Point	CUDA	OpenCL
Developed for	Nvidia GPUs only	Arbitrary GPUs and other processors (FPGAs, DSPs)
Portability	Limited to Nvidia hardware	**Highly portable**
Setup / extra code	**Relatively modest** setup	Must include a good deal of code about target systems and how to run
Code complexity	Simpler programs	More code needed
Compiler	Special compiler `nvcc` (host and device code in `.cu` file)	Not discussed in detail in the textbook
Languages	C, C++, Fortran, Python, Java APIs	Not discussed in detail in the textbook
Used in the textbook?	**Yes** (because of modest setup)	No

Why the textbook chooses CUDA

Since CUDA was developed to run only on Nvidia GPUs, it requires relatively modest setup, and as a consequence the textbook uses CUDA instead of OpenCL.

Common points

- Both are APIs for **general-purpose programming on GPUs (GPGPU)**, replacing graphics APIs.
- Both are used for **heterogeneous computing**, in which a host processor and a device processor with different architectures cooperate.

Conclusion

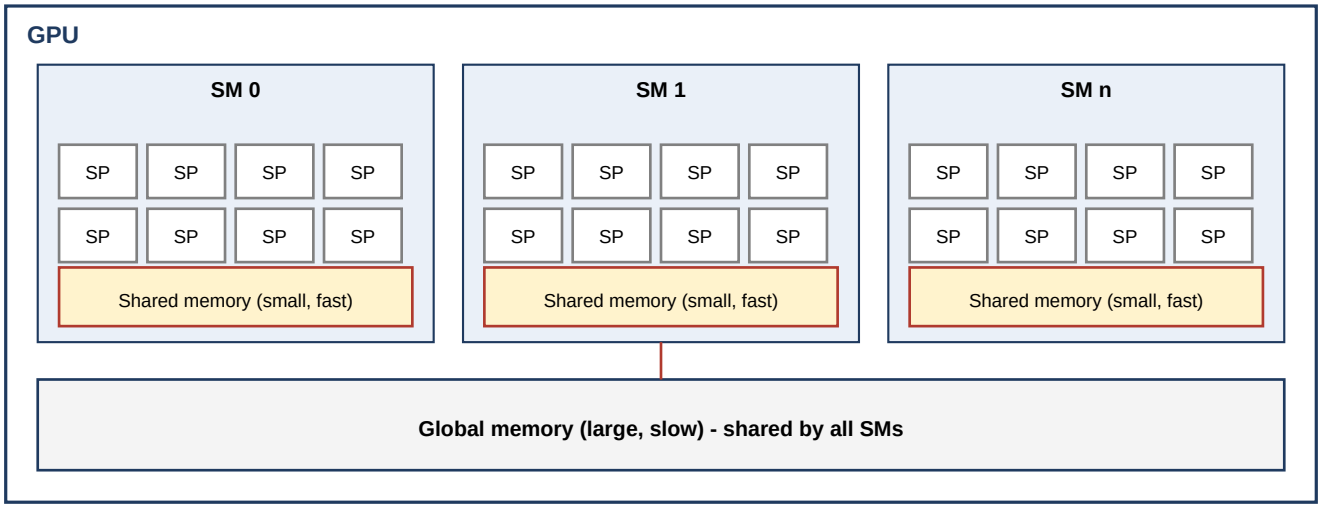
CUDA is simpler to set up but tied to Nvidia GPUs; OpenCL is portable across many devices at the cost of extra code.

Q. 10 (b) Explain Kernel with shared memory. [6 Marks]

Introduction

A **kernel** is a function that is **started by the host but runs on the device**. It is identified by the keyword `__global__` and always has return type `void`. Each thread runs a copy of the kernel (**SPMD**). Kernel calls are **asynchronous**: the host returns immediately, so `cudaDeviceSynchronize()` is used to wait.

Memory in CUDA



- Each SM has its own **shared memory**, accessible only to the SPs of that SM. More precisely, the shared memory allocated for a thread block is accessible **only to the threads in that block**.
- **Global memory** is accessible to all SPs and all threads. It is large but slow.
- Shared memory is small but quite fast.

Level	Size	Speed
Registers (top)	Smallest (e.g. 504 bytes per thread)	Fastest: about 1 cycle to copy 4 bytes
Shared memory (middle)	Small (e.g. 48 KB per block)	Up to an order of magnitude slower than registers
Global memory (bottom)	Largest (e.g. 1 GB to 12 GB)	Two to three orders of magnitude slower than registers

Local variables are stored in registers if enough storage exists; otherwise they "spill" to a thread-private region of global memory.

Declaring shared memory in a kernel

(i) Size known at compile time: `__shared__` sets aside storage in the shared memory of the SM running the block (one copy per block).

```
__shared__ float shared_vals[32];
```

(ii) Size not known at compile time: declare it `extern` and give the size in bytes as the **third argument** inside the triple angle brackets.

```
extern __shared__ float shared_vals[];
...
Dev_trap<<<blk_ct, th_per_blk, th_per_blk*sizeof(float)>>>(...args...);
```

Example: kernel function using shared memory for a sum

Threads of the same block can all access the same shared memory locations, so shared memory can be used when registers of other threads cannot be read (compute capability < 3.0, no warp shuffle). A **warp** is a set of 32 consecutive-rank threads of a block that operate in SIMD fashion, so they execute in

lockstep.

```
__device__ float Shared_mem_sum(float shared_vals[]) {
    int my_lane = threadIdx.x % warpSize;
    for (int diff = warpSize/2; diff > 0; diff = diff/2) {
        /* Make sure 0 <= source < warpSize */
        int source = (my_lane + diff) % warpSize;
        shared_vals[my_lane] += shared_vals[source];
    }
    return shared_vals[my_lane];
}
```

- Called by all threads of a warp; `shared_vals` should be in the SM's shared memory (declared `__shared__` for best performance).
- Since threads of a warp run in lockstep, there is **no race condition**: all read `shared_vals[source]` before any thread updates `shared_vals[my_lane]`.
- It is a **dissemination sum**, not a tree sum; after the loop **every thread** has the correct sum.
- If a block has more than one warp, each warp's sum is stored in a shared array `warp_sum_arr[]`, `__syncthreads()` is called, and warp 0 adds them. A warp shuffle cannot be used here because it only reads registers of threads in the **same warp**.

Shared memory banks

- Shared memory of an SM is divided into **32 banks** (16 for compute capability < 2.0) so that the 32 threads of a warp can access it simultaneously.
- If each thread in a warp accesses a **different bank**, the accesses happen **simultaneously**.
- If several threads access **different locations in the same bank**, the accesses are **serialised** (e.g. elements 0, 32, 64, ... are all in bank 0, about 32 times slower).
- If several threads read the **same location**, the value is **broadcast** and reads are simultaneous.
- So store results in a **contiguous** subarray of shared memory to use different banks.

Advantages

Shared memory is much faster than global memory, is shared by threads of a block, and the shared-memory implementation can be just as fast as the warp-shuffle (register) version.

Conclusion

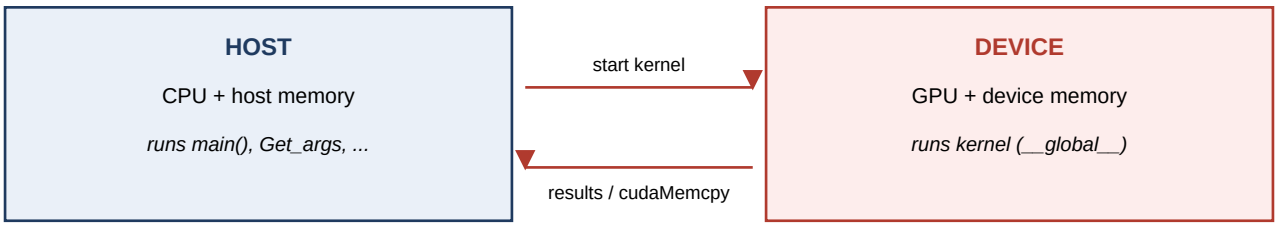
A kernel can use `__shared__` memory for fast communication between threads of a block. Correct use needs lockstep warps or `__syncthreads()`, and avoiding bank conflicts.

Q. 10 (c) Explain Heterogeneous computing in detail. [8 Marks]

Introduction

Earlier parallel programs assumed that all processors have **identical architectures**. A program that runs on a GPU is an example of **heterogeneous computing**, because it uses both a **host processor (a conventional CPU)** and a **device processor (a GPU)**, and the two have **different architectures**.

Host and device



- **Host:** the CPU together with its associated memory.
- **Device:** the GPU together with its memory.
- They are usually **physically separate**. In earlier systems, data had to be explicitly transferred between host and device memory with a function call. In recent systems (compute capability >= 3.0) explicit transfers are not needed for correctness (unified memory), though they may improve performance.

How the program is written

- A **single program** is written (**SPMD** approach), but some functions are for the CPU and some are for the GPU, so effectively **two programs** are written: a **host program** and a **device program**.
- CUDA programs cannot be compiled with an ordinary C compiler. CUDA is not just a library; the `nvcc` **compiler** must generate machine code for **two different processors**.
- File suffix is `.cu`; compile with `$ nvcc -o cuda_hello cuda_hello.cu`.

Why heterogeneous computing is important

- From about 1986 to 2003 single-thread CPU performance grew by more than **50% per year**; from 2015 to 2017 it grew by less than **4% per year**.
- So programmers look for other processors to boost performance. Besides GPUs these include **FPGAs** (programmable logic blocks and interconnects that can be configured before program execution) and **DSPs** (special circuitry for manipulating signals, e.g. compressing and filtering real-world analog signals).

Example: CUDA "hello" program

```
#include <stdio.h>
#include <cuda.h>                       /* header file for CUDA */

__global__ void Hello(void) {           /* device code: runs on GPU */
    printf("Hello from thread %d!\n", threadIdx.x);
}

int main(int argc, char* argv[]) {      /* host code: runs on CPU */
    int thread_count = strtol(argv[1], NULL, 10);
    Hello<<<1, thread_count>>>();       /* start thread_count threads on GPU */
    cudaDeviceSynchronize();            /* wait for GPU to finish */
    return 0;
}
```

Working

1. Execution begins in `main`, which runs on the **host**; it gets the number of threads from the command line.
2. The call `Hello<<<1, thread_count>>>()` starts the **kernel** on the device. `1` is the number of thread blocks and `thread_count` is the number of threads per block.
3. When the kernel starts, the system initialises `threadIdx`; each thread prints a message containing its rank `threadIdx.x`, then terminates.
4. Kernel execution is **asynchronous**: the host returns as soon as it has told the system to start the kernel. `cudaDeviceSynchronize()` forces the host to wait. Without it the program could end before any output appears.
5. The host then terminates with `return 0`.

Running `./cuda_hello 10` prints `Hello from thread 0!` to `Hello from thread 9!`.

Handling data between host and device

Unified memory	Explicit transfers
`cudaMallocManaged`; pointers valid on both host and device	`malloc` for host arrays, `cudaMalloc` for device arrays
System decides when to transfer data	Programmer copies with `cudaMemcpy` (`cudaMemcpyHostToDevice`, `cudaMemcpyDeviceToHost`)
Needs compute capability >= 3.0 and 64-bit OS; may be slower	Works on all devices; programmer can reduce transfer cost by omitting or overlapping transfers

Conclusion

Heterogeneous computing combines processors of different architectures. In CUDA, the host controls the program and starts kernels, while the GPU executes them in parallel, giving performance that is hard to get from CPUs alone.